ACTIVITAT AVALUABLE AC4

Mòdul: M0612- Desenvolupament web en entorn client

U1 – Fonaments de JavaScript

Professor: Albert Guardiola

Data límit d'entrega: 22/10/2026 23:59

Mètode d'entrega: App publicada a Github Pages (URL del repo)

Avaluació de la tasca: La nota serà la del control posterior que es realitzarà a classe (50%) més la comprovació de la funcionalitat desplegada a GitHub Pages (50%).

Resultats de l'aprenentatge:

RA1, RA2, RA3, RA4

AC4- **Dashboard d'Inventari**

L'objectiu d'aquesta pràctica és construir la lògica d'un dashboard interactiu. Treballaràs amb un array d'objectes anomenat `productes` i aprendràs a manipular-lo combinant els mètodes clàssics de JavaScript amb la **programació funcional** i les noves funcions d'**immutabilitat d'ES2023**.

Tasques Pas a Pas

Tasca 1: Mètriques del Dashboard (Programació Funcional i Accés)

Al dashboard tenim 4 targetes superiors. Has de calcular els seus valors a la funció actualitzarMetriques():

1.1 Total de l'inventari amb reduce

El mètode reduce acumula tots els elements d'un array en un únic valor final.

- **Sintaxi:** array.reduce((acumulador, element) => nouAcumulador, valorInicial) •
  Exemple de sintaxi:

```
const suma = nombres.reduce((acc, num) => acc + num, 0);
```

```
// --- TASCA 1: MÈTRIQUES DEL DASHBOARD (reduce, at, some, every) ---
function actualitzarMetriques() {
    // DONE 1.1: Utilitza 'reduce' per calcular el valor total (preu * stock)
    // L'acumulador comença a 0 i hi afegim el preu multiplicat pel stock.
    const total = productes.reduce((acc,producte) => {
        return acc + (producte.preu *producte.stock);
    }, 0);
    // acc-> Total q portem acumulat. / producte-> Objecte que estem processant en volta
    // return acc + -> Retorna nou acumulador per volta seguent
    // 0 -> valor inicial acumulador.  total-> Rep resultat final de reduce()

    document.getElementById("val-total").innerText = `${total.toFixed(2)} €`;
    // Mostrem el total amb dos decimals a la targeta de l'HTML.
```

1.2 Accés a l'últim element amb at()

El mètode at() permet accedir als elements d'un array fent servir índexs positius o negatius. L'índex -1 fa referència a l'últim element.

- **Sintaxi:** array.at(indexNegatiu)
- **Exemple de sintaxi:**

```
const ultimElement = llista.at(-1);
```

```
// DONE 1.2: Utilitza el mètode 'at(-1)' per obtenir l'últim producte afegit a l'array
// L'índex -1 permet accedir al final sense calcular la longitud.
const ultim = productes.at(-1);

// Si existeix un últim producte, mostrem el seu nom.
// Si l'array està buit, mostrem un guió.
document.getElementById("val-ultim").innerText = ultim ? ultim.nom : "-";
```

1.3 Comprovar si hi ha faltes de stock amb some

El mètode some retorna true si **almenys un** element de l'array compleix la condició.

- **Sintaxi:** array.some(element => condicio)
- **Exemple de sintaxi:**

  const hiHaNegatius = nombres.some(n => n < 0);

```
// DONE 1.3: Utilitza 'some' per comprovar si hi ha cap producte sense stock (stock === 0)
const téSenseStock = productes.some(producte => producte.stock === 0);
// some() retorna un booleà: true o false.

document.getElementById("val-sense-stock").innerText = téSenseStock ? "⚠️ Manca stock" : "✅ Ok";
```

1.4 Validar preus vàlids amb every

El mètode every retorna true només si **tots** els elements compleixen la condició.

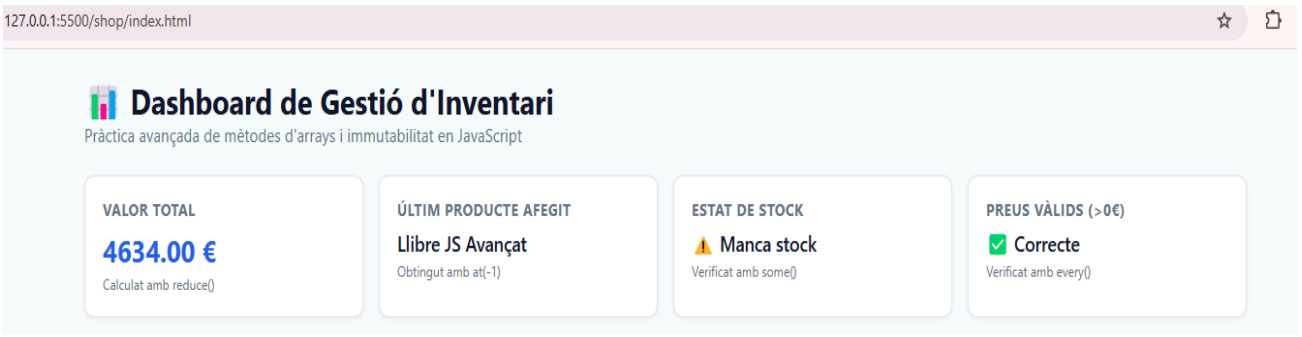
- **Sintaxi:** array.every(element => condicio)
- **Exemple de sintaxi:**

  const totsPositius = nombres.every(n => n > 0);

```
// DONE 1.4: Utilitza 'every' per comprovar si tots els preus són superiors a 0
// every() retorna false tan bon punt troba un preu que no compleix la condició.
const totsValids = productes.every(producte => producte.preu > 0 );

// Mostrem si tots els preus compleixen la condició.
document.getElementById("val-preu-valid").innerText = totsValids ? "✅ Correcte" : "❌ Error";
```

RESULTAT DEL PRIMER PUNT COMPLETAT:



Tasca 2: Afegir i Editar Productes (push, with i generació d'ID)

A l'esdeveniment submit del formulari es gestionen tant les altes com les edicions:

2.1 Generar una ID auto-incremental amb map i Math.max

Per trobar la ID més alta existent, transformem l'array d'objectes en un array de només IDs amb map i calculem el valor màxim.

- **Sintaxi:** Math.max(...array.map(element => element.propietat))
- **Exemple de sintaxi:**

```
const maxId = llista.length > 0 ? Math.max(...llista.map(x => x.id)) : 0;
```

```
} else {
    // CREAR NOU PRODUCTE
    // DONE 2.1: Calcula la nova ID trobant la màxima actual amb 'map' i 'Math.max' (si no hi ha productes, usa 0)
    const maxId = productes.length > 0 ? Math.max(...productes.map(p => p.id)) : 0;
    // map() crea un array amb les IDs i ... les passa a Math.max().
    // Si no hi ha productes, prenem 0 com a valor de partida.
    const nouProducte = { id: maxId + 1, ...dadesForm };
    // Creem el producte amb la següent ID i les dades del formulari.
```

2.2 Afegir un nou element amb push

Afegeix un element al final de l'array original (modifica l'array existent).

- **Sintaxi:** array.push(nouElement)

```
// DONE 2.2: Afegeix 'nouProducte' a l'array 'productes' amb 'push'
// ESCRIU EL TEU CODI AQUÍ
productes.push(nouProducte);
```

2.3 Modificar de manera immutable amb with (ES2023)

Si s'està editant un element, en lloc de modificar l'array directament per l'índex (array[i] = ...), el mètode with() retorna una **nova còpia de l'array** canviant només la posició indicada.

- **Sintaxi:** const nouArray = array.with(index, nouValor)
- **Exemple de sintaxi:**

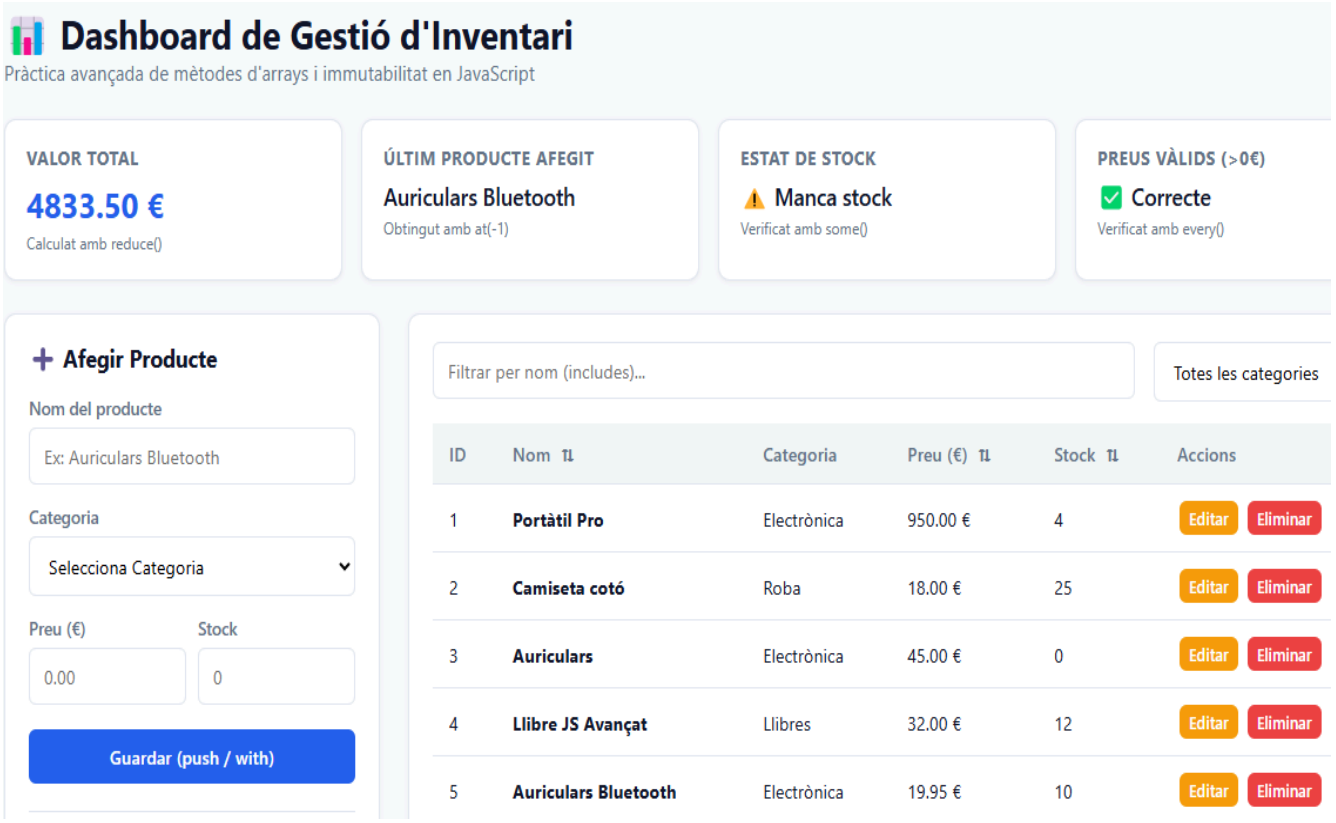
```
const llistaActualitzada = llista.with(2, "Nou Valor");
```

```
if (editId) {
    // EDITAR PRODUCTE EXISTENT
    const idNum = parseInt(editId);

    // DONE 2.3: Troba l'índex del producte amb 'findIndex'
    // Busquem la posició del producte que té aquesta ID.
    // findIndex() retorna -1 si no el troba.
    const index = productes.findIndex(p => p.id === idNum );

    if (index !== -1) {
        // Conservem la ID i incorporem les dades actualitzades del formulari.
        const producteEditat = { id: idNum, ...dadesForm };
        // DON 2.3: Actualitza l'array 'productes' utilitzant el mètode immutable 'with' (ES2023)
        // with() crea un nou array amb el producte substituït en aquesta posició.
        // // Reassignem productes perquè apunti al nou array.
        productes = productes.with(index, producteEditat);
    }
```

Resultat al afegir un producte més:



Tasca 3: Eliminar de Manera Immutable (toSpliced)

A la funció eliminarProducte(id):

3.1 Cerca de l'índex amb findIndex

Retorna la posició del primer element que compleix la condició (o -1 si no existeix).

- **Sintaxi:** const index = array.findIndex(element => condicio)

```
function eliminarProducte(id) {
    // DONE 3.1: Troba l'índex del producte amb 'findIndex'
    // Busquem la posició del producte amb aquesta ID.
    const index = productes.findIndex(p => p.id === id);
```

3.2 Eliminar sense modificar l'original amb toSpliced (ES2023)

A diferència de splice (que modifica l'array original), toSpliced retorna un **nou array** sense l'element eliminat.

- **Sintaxi:** const nouArray = array.toSpliced(indexInici, quantitatAEliminar)
- **Exemple de sintaxi:**

  const llistaSenseElement = llista.toSpliced(index, 1);

```
// Només eliminem si hem trobat el producte.
if (index !== -1) {
    // DONE 3.2: Reemplaça l'array 'productes' per un de nou sense el producte eliminat fent servir 'toSpliced' (ES2023)
    // Creem un nou array eliminant un element des de la posició trobada.
    productes = productes.toSpliced(index, 1);

    // Actualitzem la taula i les mètriques.
    actualitzarDashboard();
}
```

Tasca 4: Cerca Avançada del Darrer Element (findLast)

Al botó de cerca per categoria:

4.1 Cercar des del final amb findLast

Troba el **darrer element** de l'array que compleix una condició (recorre l'array de dreta a esquerra).

- **Sintaxi:** array.findLast(element => condicio)
- **Exemple de sintaxi:**

```
const ultimParell = nombres.findLast(n => n % 2 === 0);
```

```
// --- TASCA 4: CERCA AVANÇADA DEL DARRER ELEMENT (findLast) ---
document.getElementById("btn-cerca-last").addEventListener("click", () => {
    const cat = document.getElementById("cerca-cat-last").value;

    // DONE 4.1: Utilitza 'findLast' per trobar el darrer producte que pertanyi a la categoria 'cat'
    // Busquem des del final l'últim producte de la categoria seleccionada.
    // Si no hi ha cap coincidència, findLast() retorna undefined.
    const ultimCat = productes.findLast(p => p.categoria === cat );

    if (ultimCat) {
        alert(`Darrer producte trobat de ${cat}: ${ultimCat.nom} (${ultimCat.preu}€)`);
    } else {
        alert(`No hi ha cap producte de la categoria ${cat}`);
    }
});
```

A “Cerca Avançada” busco el últim producte d’Electrònica i borro el llibre:





Tasca 5: Ordenació per Columnes (toSorted i toReversed)

En clicar sobre les capçaleres de la taula:

5.1 Ordenar sense modificar l'original amb toSorted (ES2023)

A diferència de sort(), toSorted() retorna una còpia ordenada de l'array sense alterar l'ordre original.

- **Sintaxi:** const ordenat = array.toSorted((a, b) => comparacio)
- **Exemple de sintaxi:**

```
// Per a números:
const perPreu = productes.toSorted((a, b) => a.preu - b.preu);
// Per a cadenes de text:
const perNom = productes.toSorted((a, b) => a.nom.localeCompare(b.nom));
```

```
// DONE 5.1: Utilitza 'toSorted' (ES2023) per obtenir un array ordenat segons la columna (nom, preu o stock)
let llistaOrdenada = productes.toSorted((a, b) => {
    if (columna === "nom"){
        // Comparem els noms per ordenar-los alfabèticament.
        return a.nom.localeCompare(b,nom);
    }

    // Per a preu i stock, comparem els valors numèrics.
    return a[columna] - b[columna];
});
```

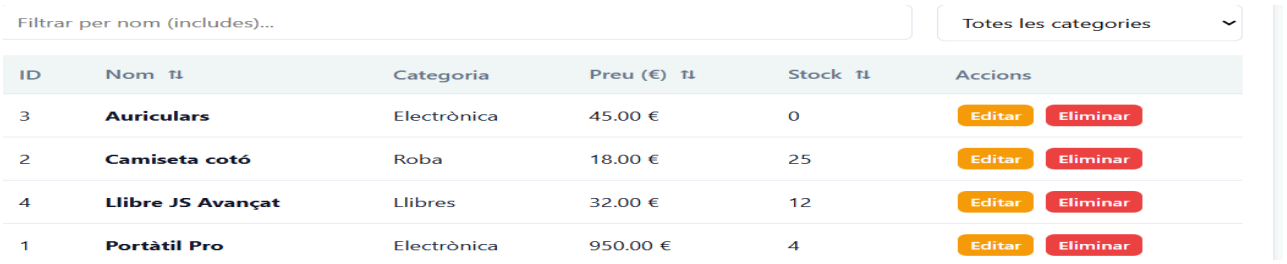
5.2 Invertir l'ordre amb toReversed (ES2023)

Retorna una nova còpia de l'array amb els elements en ordre

invers. • **Sintaxi:** const invertit = array.toReversed()

```
// DONE 5.2: Si 'estatOrdenacio.ascendent' és fals, inverteix la llista utilitzant 'toReversed' (ES2023)
if (!estatOrdenacio.ascendent) {
llistaOrdenada = llistaOrdenada.toReversed();
} // Si toca ordre descendent, invertim la còpia ordenada.

mostrarProductes(llistaOrdenada);
```



Tasca 6: Cerca i Filtres Combinats (filter i includes)

A la funció aplicarFiltres():

6.1 Filtrar per text i categoria amb filter i includes

El mètode filter retorna un subconjunt d'elements, i includes comprova si una cadena conté un text determinat.

- **Sintaxi:** array.filter(element => condicio)
- **Sintaxi includes:** cadena.includes(textCercat)
- **Exemple de sintaxi:**

```
const resultat = llista.filter(item =>
 item.nom.toLowerCase().includes(text.toLowerCase())
);
```

```
// DONE 6.1: Utilitza 'filter' i el mètode 'includes' per filtrar els productes per nom i/o categoria
const filtrats = productes.filter(p => {
    //filter() recorre els productes i conserva els que retornen true. Retorna un array nou sense modificar productes.
    // Passem el nom a minúscules per comparar sense distingir majúscules.
    const coincideixNom = p.nom.toLowerCase().includes(text);

    // "Totes" permet qualsevol categoria.
    const coincideixCategoria = cat === "Totes" || p.categoria === cat;

    // El producte només passa el filtre si compleix les dues condicions.
    return coincideixNom && coincideixCategoria;
});

mostrarProductes(filtrats);
```

Resum de la Taula de Mètodes

Mètode	Categoria	Retorna	Modifica l'original?
push()	Mutació bàsica	Nova longitud	**Sí**
findIndex()	Cerca	Índex (number)	No
includes()	Cerca de text/array	Booleà (true/false)	No
at()	Accés	Element	No
findLast()	Cerca avançada	Element / undefined	No
map()	Transformació	Nou Array	No
filter()	Filtrat	Nou Array	No
reduce()	Acumulació	Un sol valor	No
some() / every()	Comprovació	Booleà (true/false)	No
with() **(ES2023)**	Immutabilitat	Nou Array	No
toSpliced() **(ES2023)**	Immutabilitat	Nou Array	No
toSorted() **(ES2023)**	Immutabilitat	Nou Array	No
toReversed() **(ES2023)**	Immutabilitat	Nou Array	No

Pregunta 1: Diferència entre `splice()` i `toSpliced()`

Pregunta: Quin és el principal benefici d'utilitzar `toSpliced()` (introduït a ES2023) en lloc del tradicional `splice()` quan treballem amb estats d'aplicacions o frameworks moderns?

- **Resposta esperada:** `splice()` és un mètode mutant que modifica directament l'array original sobre el qual s'executa. En canvi, `toSpliced()` és un mètode immutable: deixa l'array original intacte i en retorna una còpia modificada. En programació moderna (i en gestió d'estat), mantenir la immutabilitat evita efectes secundaris (*side effects*) no desitjats.

Pregunta 2: Comportament de `some()` vs `every()`

Pregunta: Quin serà el resultat de les següents dues línies d'execució si tenim un array de productes on un d'ells té preu `0` i la resta tenen preus positius?

JavaScript
```
const res1 = productes.some(p => p.preu === 0);
const res2 = productes.every(p => p.preu > 0);
```

- **A)** `res1 = true`, `res2 = true`
- **B)** `res1 = false`, `res2 = true`
- **C)** `res1 = true`, `res2 = false`
- **D)** `res1 = false`, `res2 = false`

- **Explicació:** `some()` retorna `true` perquè **almenys un** element compleix la condició (`preu === 0`). `every()` retorna `false` perquè **no tots** els elements compleixen la condició (`preu > 0`).

Pregunta 3: Rendiment de cerca amb `findLast()`

Pregunta: Si tenim una llista de 10.000 registres ordenats cronològicament (d'antics a nous) i volem obtenir la transacció més recent d'un usuari concret, per què és més eficient utilitzar `findLast()` que `filter()` o `find()`?

- **Resposta esperada:**
  1. Respecte a `filter()`: `filter()` ha de recórrer **tots** els 10.000 elements per construir un nou array, mentre que `findLast()` s'atura tan bon punt troba la primera coincidència.
  2. Respecte a `find()`: `find()` comença a cercar per l'inici (registres més antics). En canvi, `findLast()` comença a cercar des del final (registres més recents), per la qual cosa trobarà la transacció més recent molt més ràpidament sense haver de recórrer tota la llista antiga.

Pregunta 4: Mètodes immutables d'ES2023

Pregunta: Volem canviar el valor del tercer element d'un array de forma **immutable** (sense modificar l'array original). Quina de les següents opcions és la correcta utilitzant les funcions d'ES2023?

- **A)** `array[2] = "Nou Valor";`
- **B)** `const nouArray = array.with(2, "Nou Valor");`
- **C)** `const nouArray = array.splice(2, 1, "Nou Valor");`
- **D)** `const nouArray = array.at(2, "Nou Valor");`

- **Explicació:** `with(index, valor)` és el mètode dissenyat específicament a ES2023 per retornar una còpia d'un array actualitzant un element en una posició determinada de manera immutable.

Pregunta 5: Valor inicial en el mètode `reduce()`

Pregunta: Què passa si executem el mètode `reduce()` per calcular el valor total d'un inventari sense passar-li el segon argument (el valor inicial `0`), com en aquest codi?

```
const total = productes.reduce((acc, p) => acc + (p.preu * p.stock));
```

- **Resposta esperada:** El codi fallarà o retornarà un resultat incorrecte (`NaN`). Si no es proporciona un valor inicial a `reduce()`, el primer element de l'array s'assigna com a `acc` inicial (que és un objecte enter `{ id: 1, preu: ... }`) i la iteració comença des del segon element. En intentar sumar un objecte amb un número (`objecte + number`), la dada es corromp. Per tant, quan s'acumulen propietats d'un array d'objectes, el valor inicial (ex: `0`) és obligatori.